

Angular

COMPONENTE = è una semplice classe **TypeScript** collegata a un tag HTML personalizzato (chiamato *selector*). (una sorta di template)

Come si crea:

ng generate component 'nome-del-componente' → vengono generati l'html, il css e il .ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-card-prodotti',
  imports: [],
  template: ``,
  style: ``,
})
export class CardProdotti {}
```

Come si usa? → con il selector → <app-card-prodotti></app-card-prodotti>, mettendolo negli imports

TEMPLATING

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-card-prodotti',
  imports: [],
  template: `
    <h2>{{nomeProdotto}}</h2>
  `,
  style: ``,
})
export class CardProdotti {}
```

```

    nomeProdotto = 'Jordan Retro 3'
  }

```

Per mandare a schermo la variabile si inserisce tra {{ }}.

PROPERTY BINDING = collegare una variabile con una proprietà html.

```

import { Component } from '@angular/core';

@Component({
  selector: 'app-card-prodotti',
  imports: [],
  template: `
    <h2>{{nomeProdotto}}</h2>
    <button [disabled]='isDisabled'>Cliccami</button> //c
on le [] si connette una proprietà ad una variabile
  `,
  style: ``,
})
export class CardProdotti {
  nomeProdotto = 'Jordan Retro 3'
  isDisabled = true
}

```

EVENT BINDING

```

import { Component } from '@angular/core';

@Component({
  selector: 'app-card-prodotti',
  imports: [],
  template: `
    <h2>{{nomeProdotto}}</h2>
    <button [disabled]='isDisabled' (click)='salutami()'>
Cliccami</button>
  `,
})

```

```

        style: ` `,
    })
    export class CardProdotti {
        nomeProdotto = 'Jordan Retro 3'
        isDisabled = true

        salutami(){
            this.nomeProdotto = 'GTA 6'
            console.log('ti ho salutato')
        }
    }

```

2 WAY BINDINGS

```

import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms'; //per far funzionare ngModel bisogna importare FormModule da angular/forms

@Component({
    selector: 'app-card-prodotti',
    imports: [FormsModule], //per far funzionare ngModel bisogna importarlo anche qua
    template: `
        <h2>{{nomeProdotto}}</h2>
        <button [disabled]='isDisabled' (click)='salutami()'>
        Cliccami</button>
        <input [(ngModel)]='nome' placeholder='Il tuo nome'>
    </input> //si mette sia () che []
    `,
    style: ` `,
})
export class CardProdotti {
    nomeProdotto = 'Jordan Retro 3'
    isDisabled = true
    nome = 'Riccardo'

```

```

    salutammi(){
      this.nomeProdotto = this.nome
      console.log('ti ho salutato: ', this.nome)
    }
  }
}

```

CONTROL FLOW = definire il comportamento del componente in base alle situazioni

```

import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
@Component({
  selector: 'app-card-prodotti',
  imports: [FormsModule],
  template: `
    <h2>{{nomeProdotto}}</h2>
    <button [disabled]='isDisabled' (click)='salutammi() '>
    Cliccami</button>
    <input [(ngModel)]='nome' placeholder='Il tuo nome'>
  </input>

    @if (isDisabled) {
      //control flow, si pot
      rebbe fare anche uno @switch () { @case () }
      <p>è disabilitato</p>
    } @else {
      <p>non è disabilitato</p>
    }

    @for (item of items; track $index) {
      <p>{{$index + 1}} {{item}}</p> //track tiene tra
      ccia dell'indice
    }

    `
  ,
  style: ``,
})

```

```
export class CardProdotti {
  nomeProdotto = 'Jordan Retro 3'
  isDisabled = true
  nome = 'Riccardo'
  items = ['ciao', 'ciao1', 'ciao2']

  salutami(){
    this.nomeProdotto = this.nome
    console.log('ti ho salutato: ', this.nome)
  }
}
```

SIGNAL = ogni variabile che cambia valore e deve essere mandati a schermo è un segnale.

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
@Component({
  selector: 'app-card-prodotti',
  imports: [FormsModule],
  template: `

    <button [disabled]='isDisabled' (click)='salutami() '>
    Cliccami</button>
    <input [(ngModel)]='nome' placeholder='Il tuo nome'>
  </input>
    <h2>{{ nomeProdotto() }}</h2> //con signal si metton
    o le tonde e diventa una funzione

    `
  ,
  style: ` `,
})
export class CardProdotti {
  nomeProdotto = signal ('Jordan Retro 3') //signal
  isDisabled = signal(true)
```

```

nome = signal ('Riccardo')
items = signal(['ciao', 'ciao1', 'ciao2'])

salutami(){
  // this.nomeProdotto = this.nome non si puo fare più
  così
  this.nomeProdotto.set(this.nome()) //ma così per sost
  ituire il valore
  this.nomeProdotto.update((valorePrecedente) => valore
  Precedente + this.nome()) //con update
  console.log('ti ho salutato: ', this.nome)
}

constructor(){
  effect(() => {
    console.log('nomeProdotto è cambiato in:', this.n
    omeProdotto())
  })
}
//effect manda il console log quando il signal di nomePro
dotto è cambiato
//a differenza di prima che veniva sempre stampato a sche
rmo
}

```